

Database System

Instructor: Yiwen Shen, Ph.D.

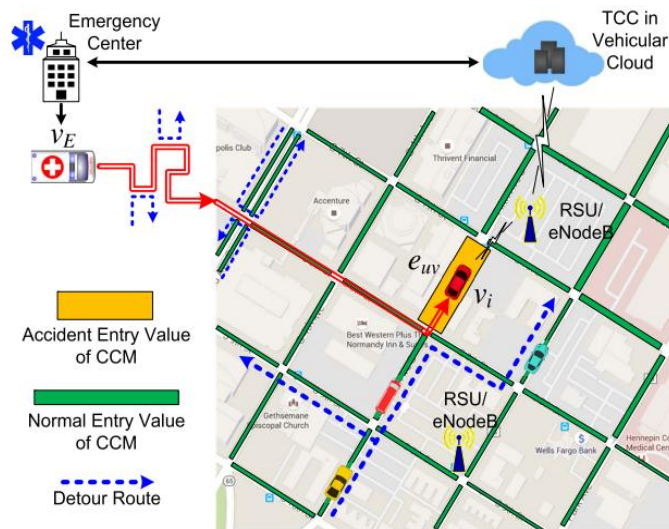
Dept. of Software & Computer Engineering,

Ajou University, Korea

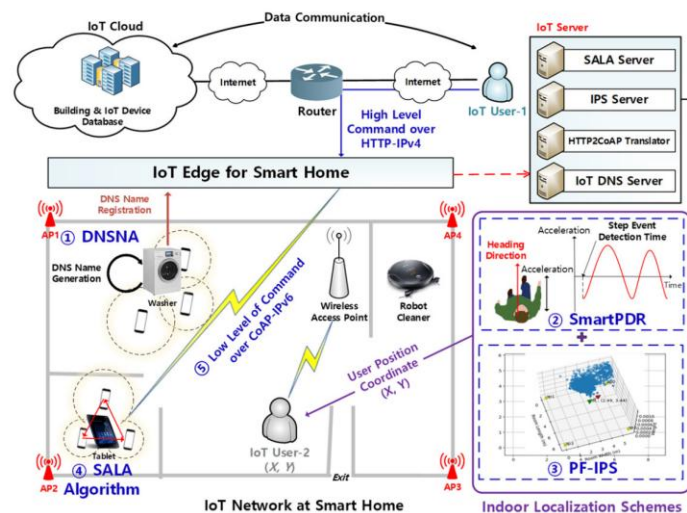
chrisshen@ajou.ac.kr

About the Instructor

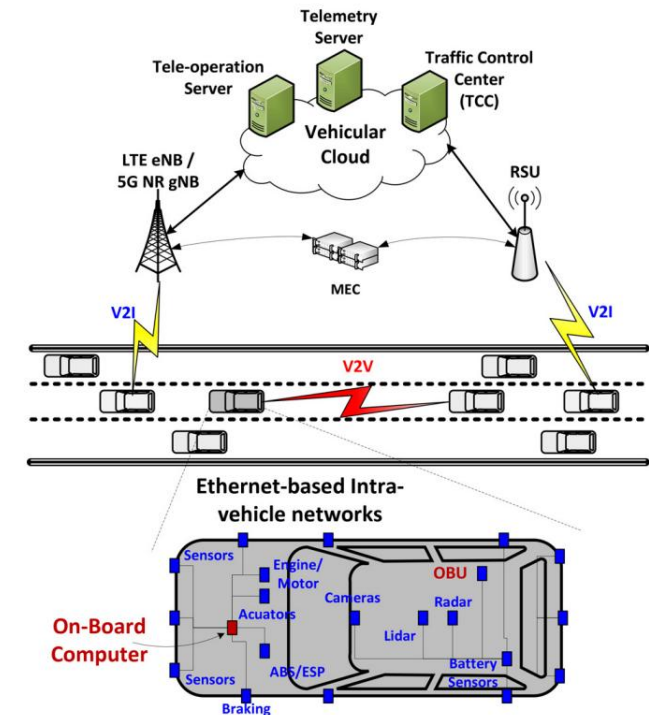
- Research interests:



Intelligent Transportation Systems (ITS)



IoT and Localization Systems



Connected and Automated Vehicles (CAV)

- Personal Homepage: <https://chrisshen.github.io/>

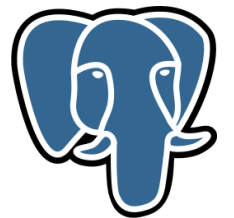
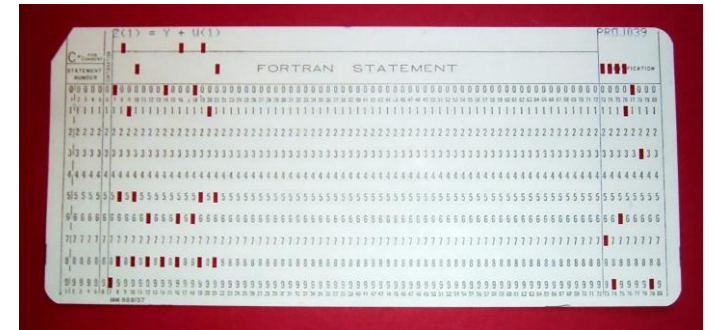
Outline

- 数据库系统历史
- 数据库系统及其应用
- 数据库系统的目的
- 数据视图
- 数据库语言
- 数据库设计
- 数据库引擎
- **Database Architecture**
- **Database Users and Administrators**

History of Database Systems



- **1950s and early 1960s:**
 - Data processing using magnetic tapes for storage
 - Tapes provided only sequential access
 - Punched cards for input
- **Late 1960s and 1970s:**
 - Hard disks allowed direct access to data
 - Network and hierarchical data models in widespread use
 - **Ted Codd** defines the **relational data model**
 - Would win the ACM Turing Award for this work
 - IBM Research begins **System R** prototype
 - UC Berkeley (Michael Stonebraker) begins **Ingres** prototype (Now **PostgreSQL**)
 - **Oracle** releases first commercial relational database
 - High-performance (for the era) transaction processing



History of Database Systems (Cont.)

- **1980s:**
 - **Research relational prototypes evolve into commercial systems**
 - SQL becomes industrial standard
 - **Parallel and distributed database systems**
 - Wisconsin, IBM, Teradata
 - **Object-oriented database systems**
- **1990s:**
 - **Large decision support and data-mining applications**
 - **Large multi-terabyte data warehouses**
 - **Emergence of Web commerce**

History of Database Systems (Cont.)

- **2000s**
 - **Big data storage systems**
 - Google BigTable, Yahoo PNuts, Amazon,
 - “NoSQL” systems.
 - **Big data analysis: beyond SQL**
 - Map reduce and friends
- **2010s**
 - **SQL reloaded**
 - SQL front end to Map Reduce systems
 - Massively parallel database systems
 - Multi-core main-memory databases

Database Ecosystem

SQL

Traditional RDBMSs



Generative Value

"Modern" SQL DBs



Not an exhaustive list of companies/segments.
*NoSQL refers to "not-only" SQL - some databases could be in multiple categories.

OLAP Database



Data Warehouse



NoSQL

Document



Graph



Vector



Time-Series*



Search



Key-Value



Multi-Model



Wide Column



中国数据库系统厂商



资料来源：《中国数据库行业研究报告》，艾瑞咨询（2021）、华泰研究

Rank			DBMS	Database Model	Score		
Aug 2025	Jul 2025	Aug 2024			Aug 2025	Jul 2025	Aug 2024
1.	1.	1.	Oracle	Relational, Multi-model ⓘ	1220.70	+3.64	-37.78
2.	2.	2.	MySQL	Relational, Multi-model ⓘ	915.46	-25.26	-111.40
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model ⓘ	754.15	-16.99	-61.02
4.	4.	4.	PostgreSQL	Relational, Multi-model ⓘ	671.25	-9.63	+33.87
5.	5.	5.	MongoDB 🟡	Document, Multi-model ⓘ	395.58	-8.25	-25.40
6.	6.	📈 7.	Snowflake	Relational	178.90	+2.73	+42.93
7.	7.	📉 6.	Redis	Key-value, Multi-model ⓘ	147.19	-2.53	-5.52
8.	8.	📈 9.	IBM Db2	Relational, Multi-model ⓘ	127.31	-0.20	+4.30
9.	📈 12.	📈 15.	Databricks	Multi-model ⓘ	115.82	+7.78	+31.36
10.	📉 9.	📉 8.	Elasticsearch	Multi-model ⓘ	114.27	-4.56	-15.56
11.	📉 10.	📉 10.	SQLite	Relational	112.59	-2.84	+7.80
12.	📉 11.	📉 11.	Apache Cassandra	Wide column, Multi-model ⓘ	108.51	-0.24	+11.51
13.	13.	📈 14.	MariaDB 🟡	Relational, Multi-model ⓘ	93.59	-1.85	+7.06
14.	14.	📉 12.	Microsoft Access	Relational	87.76	-2.71	-8.61
15.	15.	📈 17.	Amazon DynamoDB	Multi-model ⓘ	83.48	-0.67	+14.57
16.	16.	16.	Microsoft Azure SQL Database	Relational, Multi-model ⓘ	75.84	-0.67	+0.81
17.	17.	📈 19.	Apache Hive	Relational	71.03	-4.71	+15.79
18.	18.	📉 13.	Splunk	Search engine	69.77	+0.25	-26.33
19.	19.	📉 18.	Google BigQuery	Relational	65.18	+0.99	+9.65
20.	20.	📈 21.	Neo4j	Graph	54.47	-0.16	+10.58

Source: <https://db-engines.com/en/ranking>

DB-Engines Ranking

Trend:
https://db-engines.com/en/ranking_trend

Related Top-Tier Conf.

- **ACM SIGMOD (Special Interest Group on Management of Data):**
<https://sigmod.org/>
- **ACM SIGIR (Special Interest Group on Information Retrieval):**
<https://sigir.org/>
- **VLDB (Very Large Data Bases):** <https://www.vldb.org/>
 - 2025: <https://vldb.org/2025/?program-schedule-2025>
 - 2026: <https://vldb.org/2026/>
- **ACM CIKM (Conference on Information and Knowledge Management):**
 - <http://www.cikmconference.org/>
 - 2024: <https://cikm2024.org/>
 - 2025: <https://cikm2025.org/>

Database Systems

- **DBMS** (Database Management System) contains information about a particular enterprise
 - Collection of interrelated **data**
 - Set of **programs** to access the data
 - **An environment** that is both convenient and efficient to use
- Database systems are used to manage collections of data that are:
 - Highly **valuable**
 - Relatively **large**
 - Accessed by **multiple** users and applications, often at the same time.
- A modern database system is a **complex software system** whose task is to manage a large, complex collection of data.
- Databases touch all aspects of our lives

A Day In Susan's Life

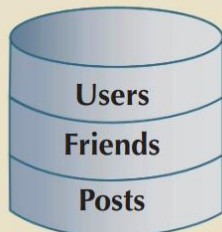
See how many databases she interacts with each day

*Before leaving for work,
Susan checks her
Facebook and
Twitter accounts*



Where is the data about the friends and groups stored?

Where are the "likes" stored and what would they be used for?



*On her lunch break,
she picks up her
prescription at the
pharmacy*



Where is the pharmacy inventory data stored?

What data about each product will be in the inventory data?

What data is kept about each customer and where is it stored?



*After work, Susan
goes to the grocery
store*



Where is the product data stored?

Is the product quantity in stock updated at checkout?

Does she pay with a credit card?



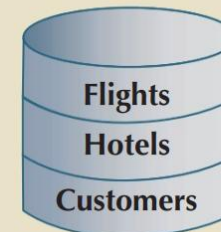
*At night, she plans for a trip
and buys airline tickets and
hotel reservations online*



Where does the online travel website get the airline and hotel data from?

What customer data would be kept by the website?

Where would the customer data be stored?



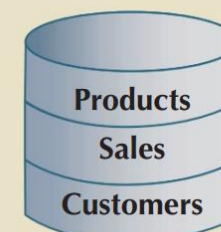
*Then she makes a few
online purchases*



Where are the product and stock data stored?

Where does the system get the data to generate product "recommendations" to the customer?

Where would credit card information be stored?



Database Applications Examples

- **Enterprise Information**
 - Sales: customers, products, purchases
 - Accounting: payments, receipts, assets
 - Human Resources: Information about employees, salaries, payroll taxes.
- **Manufacturing**: management of production, inventory, orders, supply chain.
- **Banking and finance**
 - Customer information, accounts, loans, and banking transactions.
 - Credit card transactions
 - Finance: sales and purchases of financial instruments (e.g., stocks and bonds; storing real-time market data
- **Universities**: registration, grades

Database Applications Examples (Cont.)

- **Airlines:** reservations, schedules
- **Telecommunication:** records of calls, texts, and data usage, generating monthly bills, maintaining balances on prepaid calling cards
- **Web-based services**
 - Online retailers: order tracking, customized recommendations
 - Online advertisements
- **Document databases**
- **Navigation systems:** For maintaining the locations of various places of interest along with the exact routes of roads, train systems, buses, etc.

Purpose of Database Systems

- In the **early** days, database applications were built directly on top of **file systems**, which leads to:
 - Data **redundancy** and **inconsistency**: data is stored in multiple file formats resulting in duplication of information in different files
 - **Difficulty in accessing data**
 - Need to write a new program to carry out each new task
 - Data **isolation**: Multiple files and formats
 - **Integrity** problems
 - Integrity constraints (e.g., account balance > 0) become “buried” in program code rather than being stated explicitly
 - Hard to add new constraints or change existing ones

Purpose of Database Systems (Cont.)

- **Atomicity** of updates
 - Failures may leave database in an **inconsistent** state with partial updates carried out
 - Example: Transfer of funds from one account to another should either complete or not happen at all
- **Concurrent** access by multiple users
 - Concurrent access needed for performance
 - Uncontrolled concurrent accesses can lead to inconsistencies
 - Ex: Two people reading a balance (say 100) and updating it by withdrawing money (say 50 each) at the same time
- **Security** problems
 - Hard to provide user access to some, but not all, data
- Database systems offer solutions to **all** the above problems

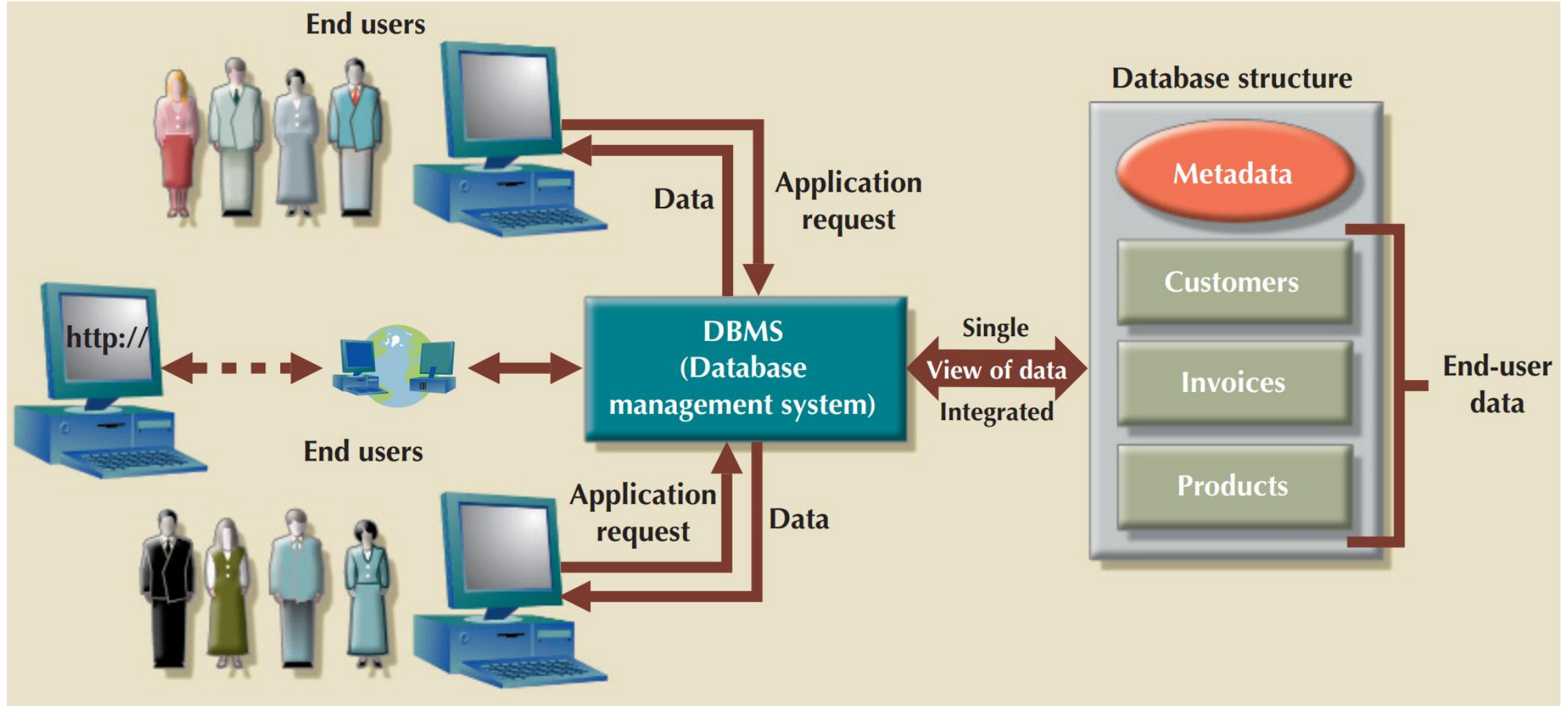
University Database Example

- In this text we will be using a university database to illustrate all the concepts
- **Data** consists of information about:
 - Students
 - Instructors
 - Classes
- **Application program** examples:
 - Add new students, instructors, and courses
 - Register students for courses, and generate class rosters
 - Assign grades to students, compute grade point averages (GPA) and generate transcripts

View of Data

- A database system is a collection of interrelated data and a set of programs that allow users to access and modify these data.
- A major purpose of a database system is to provide users with **an abstract view** of the data.
 - **Data models**
 - A collection of conceptual tools for describing data, data relationships, data semantics, and consistency constraints.
 - **Data abstraction**
 - Hide the complexity of data structures to represent data in the database from users through several levels of data abstraction.

Interaction Between End User and Database



Data Models

- **Relational model**
- **Entity-Relationship data model (mainly for database design)**
- **Object-based data models (Object-oriented and Object-relational)**
- **Semi-structured data model (XML)**
- **Other older models:**
 - **Network model**
 - **Hierarchical model**

Data Models List

- **Relational** ← **Most DBMSs** ← **Our focus**
- **Key/Value** ← **Simple Apps / Caching**
- **Graph**
- **Document / JSON / XML / Object** ← **NoSQL**
- **Wide-Column / Column-family**
- **Array (Vector, Matrix, Tensor)** ← **ML/Science**
- **Hierarchical**
- **Network**
- **Semantic** ← **Obsolete / Legacy / Rare**
- **Entity-Relationship**

Early DBMSs

- **Ted Codd** was a mathematician at IBM Research in the late 1960s.
- Codd saw IBM's developers rewriting database programs every time the database's schema or layout changed.
- Devised the relational model in 1969.
 - <https://dl.acm.org/doi/10.1145/1558334.1558336>
 - <https://dl.acm.org/doi/10.1145/362384.362685>

DERIVABILITY, REDUNDANCY AND CONSISTENCY OF RELATIONS STORED IN LARGE DATA BANKS

E. F. Codd
Research Division
San Jose, California

ABSTRACT: The large, integrated data banks of the future will contain many relations of various degrees in stored form. It will not be unusual for this set of stored relations to be redundant. Two types of redundancy are defined and discussed. One type may be employed to improve accessibility of certain kinds of information which happen to be in great demand. When either type of redundancy exists, those responsible for control of the data bank should know about it and have some means of detecting any "logical" inconsistencies in the total set of stored relations. Consistency checking might be helpful in tracking down unauthorized (and possibly fraudulent) changes in the data bank contents.

RJ 599 (# 12343)
August 19, 1969

A Relational Model of Data for Large Shared Data Banks

E. F. Codd
IBM Research Laboratory, San Jose, California

Future users of large data banks must be protected from having to know how the data is organized in the machine (the internal representation). A prompting service which supplies such information is not a satisfactory solution. Activities of users at terminals and most application programs should remain unaffected when the internal representation of data is changed and even when some aspects of the external representation are changed. Changes in data representation will often be needed as a result of changes in query, update, and report traffic and natural growth in the types of stored information.

Existing noninferential, formatted data systems provide users with tree-structured files or slightly more general network models of the data. In Section 1, inadequacies of these models are discussed. A model based on n -ary relations, a normal form for data base relations, and the concept of a universal data sublanguage are introduced. In Section 2, certain operations on relations (other than logical inference) are discussed and applied to the problems of redundancy and consistency in the user's model.

KEY WORDS AND PHRASES: data bank, data base, data structure, data organization, hierarchies of data, networks of data, relations, derivability, redundancy, consistency, composition, join, retrieval language, predicate calculus, security, data integrity
CR CATEGORIES: 3.70, 3.73, 3.75, 4.20, 4.22, 4.29

1. Relational Model and Normal Form

1.1. INTRODUCTION

This paper is concerned with the application of elementary relation theory to systems which provide shared access to large banks of formatted data. Except for a paper by Childs [1], the principal application of relations to data systems has been to deductive question-answering systems. Levein and Maron [2] provide numerous references to work in this area.

In contrast, the problems treated here are those of *data independence*—the independence of application programs and terminal activities from growth in data types and changes in data representation—and certain kinds of *data inconsistency* which are expected to become troublesome even in nondeductive systems.

The relational view (or model) of data described in Section 1 appears to be superior in several respects to the graph or network model [3, 4] presently in vogue for non-inferential systems. It provides a means of describing data with its natural structure only—that is, without superimposing any additional structure for machine representation purposes. Accordingly, it provides a basis for a high level data language which will yield maximal independence between programs on the one hand and machine representation and organization of data on the other.

A further advantage of the relational view is that it forms a sound basis for treating derivability, redundancy, and consistency of relations—these are discussed in Section 2. The network model, on the other hand, has spawned a number of confusions, not the least of which is mistaking the derivation of connections for the derivation of relations (see remarks in Section 2 on the "connection trap").

Finally, the relational view permits a clearer evaluation of the scope and logical limitations of present formatted data systems, and also the relative merits (from a logical standpoint) of competing representations of data within a single system. Examples of this clearer perspective are cited in various parts of this paper. Implementations of systems to support the relational model are not discussed.

1.2. DATA DEPENDENCIES IN PRESENT SYSTEMS

The provision of data description tables in recently developed information systems represents a major advance toward the goal of data independence [5, 6, 7]. Such tables facilitate changing certain characteristics of the data representation stored in a data bank. However, the variety of data representation characteristics which can be changed *without logically impairing some application programs* is still quite limited. Further, the model of data with which users interact is still cluttered with representational properties, particularly in regard to the representation of collections of data (as opposed to individual items). Three of the principal kinds of data dependencies which still need to be removed are: ordering dependence, indexing dependence, and access path dependence. In some systems these dependencies are not clearly separable from one another.

1.2.1. Ordering Dependence. Elements of data in a data bank may be stored in a variety of ways, some involving no concern for ordering, some permitting each element to participate in one ordering only, others permitting each element to participate in several orderings. Let us consider those existing systems which either require or permit data elements to be stored in at least one total ordering which is closely associated with the hardware-determined ordering of addresses. For example, the records of a file concerning parts might be stored in ascending order by part serial number. Such systems normally permit application programs to assume that the order of presentation of records from such a file is identical to (or is a subordering of) the

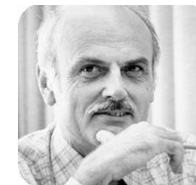
Relational Model 关系模型

- All the data is stored in various tables.
- Example of tabular data in the relational model

Columns

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Rows



Ted Codd
Turing Award 1981

https://en.wikipedia.org/wiki/Edgar_F._Codd

A Sample Relational Database

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

Levels of Abstraction

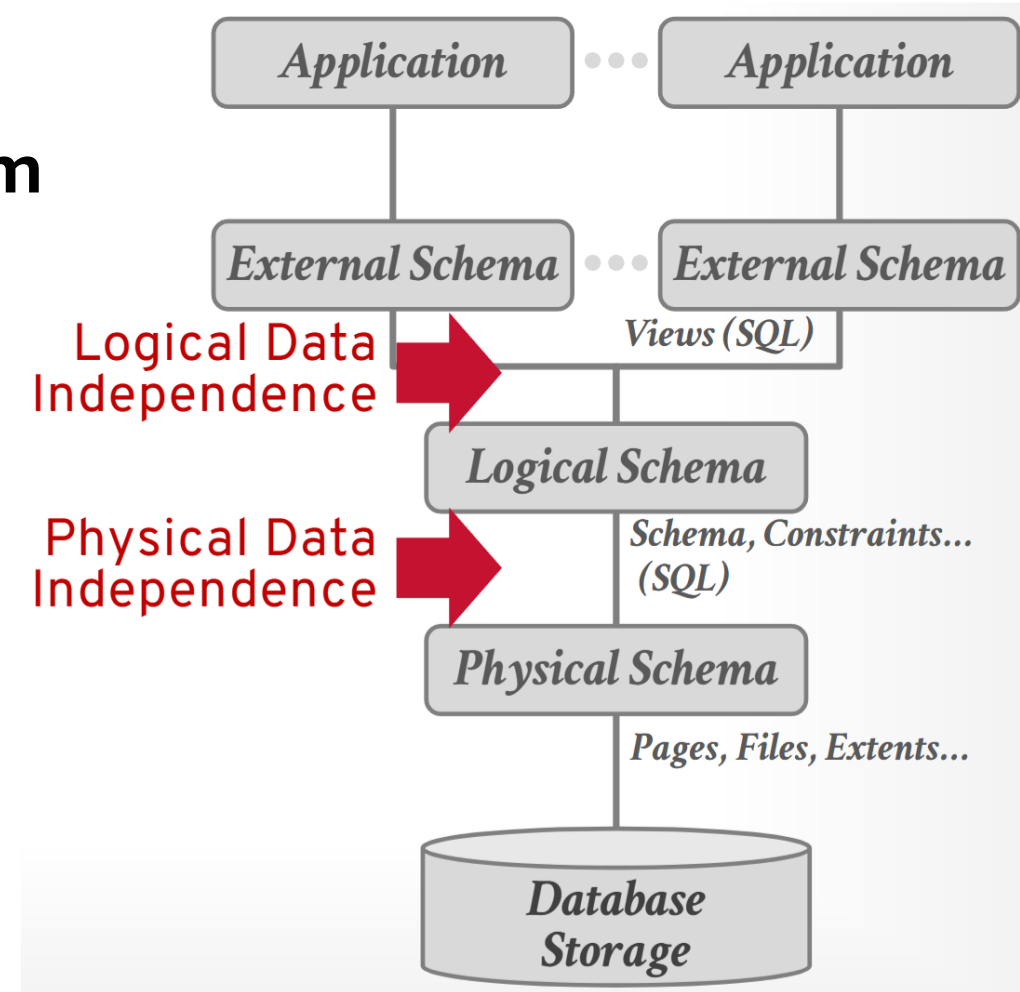
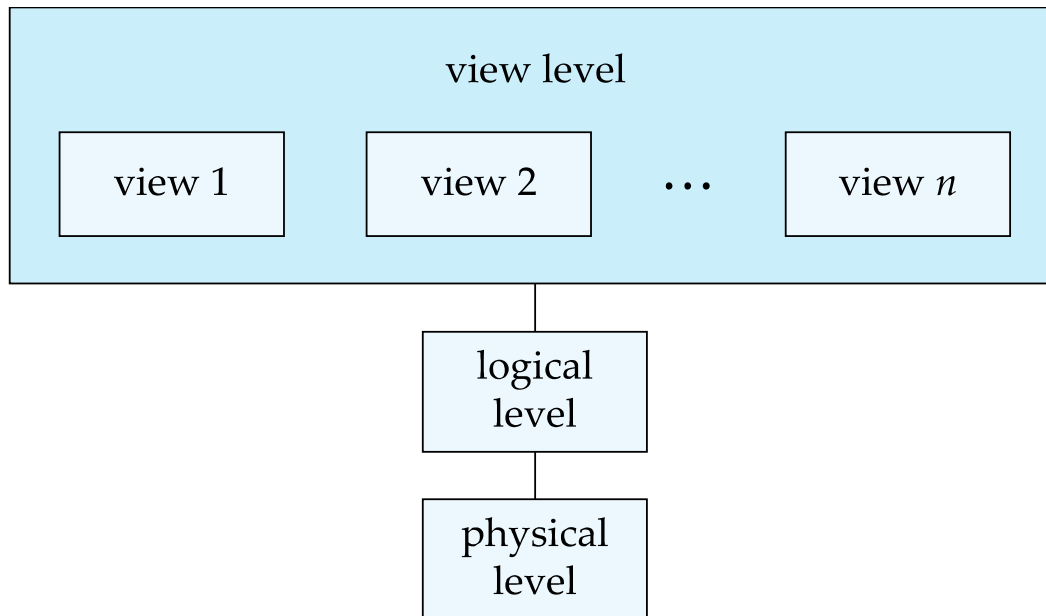
- **Physical level**: describes how a record (e.g., instructor) is stored.
- **Logical level**: describes data stored in database, and the relationships among the data.

```
type instructor = record
    ID : string;
    name : string;
    dept_name : string;
    salary : integer;
end;
```

- **View level**: application programs hide details of data types. Views can also hide information (such as an employee's salary) for security purposes.

View of Data

- An architecture for a database system



Instances and Schemas

- Similar to types and variables in programming languages
- **Logical Schema** – the overall **logical structure** of the database
 - Example: The database consists of information about a set of customers and accounts in a bank and the relationship between them
 - Analogous to type information of a variable in a program
- **Physical Schema** – the overall **physical structure** of the database
- 实例 • **Instance** – the actual content of the database at a particular point in time
 - Analogous to the value of a variable

Physical Data Independence

物理数据独立性

- **Physical Data Independence – the ability to modify the physical schema without changing the logical schema**
 - Applications depend on the logical schema
 - In general, the interfaces between the various levels and components should be well defined so that changes in some parts do not seriously influence others.

Data Definition Language (DDL)

数据定义语言

- Specification notation for **defining the database schema**

Example:

```
create table instructor (  
    ID                char(5),  
    name              varchar(20),  
    dept_name         varchar(20),  
    salary             numeric(8,2))
```

编译器

数据字典

- DDL **compiler** generates a set of table templates stored in **a data dictionary**
- **Data dictionary** contains **metadata** (i.e., data about data)
 - Database schema
 - Integrity constraints 完整性约束
 - Primary key (ID uniquely identifies instructors)
 - Authorization 授权
 - Who can access what

Data Manipulation Language (DML)

数据操作语言

- Language for accessing and updating the data organized by the appropriate data model

- DML also known as **query language**

- There are basically two types of data-manipulation language

过程 DML • **Procedural DML** -- require a user to specify what data are needed and how to get those data.

声明式 DML • **Declarative DML** -- require a user to specify what data are needed without specifying how to get those data.

- **Declarative** DMLs are usually easier to learn and use than are procedural DMLs.
- **Declarative** DMLs are also referred to as **non-procedural DMLs**
- The portion of a DML that involves **information retrieval** is called a **query language**.

SQL Query Language

- **SQL (Structured Query Language) query language is nonprocedural.** A query takes as input several tables (possibly only one) and always returns a single table.
- Example to find all instructors in Comp. Sci. dept

```
select name from instructor  
where dept_name = 'Comp. Sci.'
```
- SQL is **NOT** a Turing machine equivalent language
- To be able to compute complex functions SQL is usually embedded in some higher-level language
- Application programs generally access databases through one of
 - Language extensions to allow embedded SQL
 - Application program interface (e.g., ODBC/JDBC) which allow SQL queries to be sent to a database

Open Database Connectivity

Database Access from Application Program

- Non-procedural query languages such as SQL are not as powerful as a universal Turing machine.
 - SQL does not support actions such as input from users, output to displays, or communication over the network.
- Such computations and actions must be written in a **host language**, such as C/C++, Java or Python, with embedded SQL queries that access the data in the database.
- **Application programs** -- are programs that are used to interact with the database in this fashion.

Examples of SQL in Host Languages

• Python

```
import mysql.connector

mydb = mysql.connector.connect(
    host="localhost",
    user="yourusername",
    password="yourpassword"
)

mycursor = mydb.cursor()

mycursor.execute("CREATE
DATABASE mydatabase")
```

• C++

```
MYSQL *mysql_init(MYSQL *);

MYSQL mysql_real_connect(
    MYSQL connection,
    const char *host,
    const char *username,
    const char *password,
    const char *database_name,
    unsigned int port,
    const char *unix_socket_name,
    unsigned int flags
);

int mysql_query(MYSQL *connection, const char
*query);
```

Database Design

- The process of designing the general structure of the database:
- **Logical Design** – Deciding on the database schema. Database design requires that we find a “good” collection of relation schemas.
 - Business decision – What attributes should we record in the database?
 - Computer Science decision – What relation schemas should we have and how should the attributes be distributed among the various relation schemas?
- **Physical Design** – Deciding on the physical layout of the database

Database Engine

- A database system is **partitioned** into modules that deal with each of the responsibilities of the overall system.
- The **functional** components of a database system can be divided into
 - The **storage manager**, 存储管理器
 - The **query processor** component, 查询处理组件
 - The **transaction management** component.

事务管理组件

Storage Manager 存储管理器

- A program module that provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system.
- The storage manager is responsible to the following tasks:
 - **Interaction** with the **OS file manager**
 - Efficient **storing**, **retrieving** and **updating** of data
- The storage manager components include:
 - Authorization and integrity manager
 - Transaction manager
 - File manager
 - Buffer manager

Storage Manager (Cont.)

- The storage manager implements several data structures as part of the physical system implementation:
 - Data files -- store the database itself
 - Data dictionary -- stores metadata about the structure of the database, in particular the schema of the database.
 - Indices -- can provide fast access to data items. A database index provides **pointers** to those data items that hold a particular value.

Query Processor

查询处理器

- The query processor components include:

DDL解释器 • **DDL interpreter** -- interprets DDL statements and records the definitions in the data dictionary.

DML编译器 • **DML compiler** -- translates DML statements in a query language into an evaluation plan consisting of low-level instructions that the query evaluation engine understands.

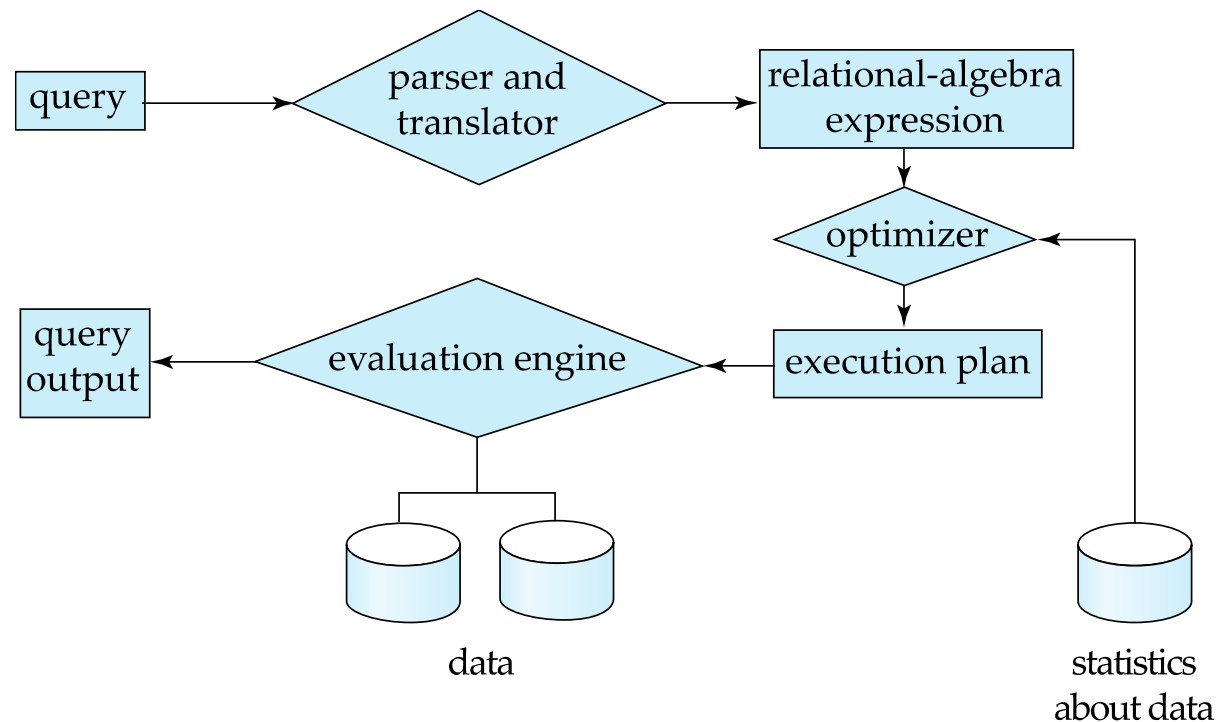
- The DML compiler performs query optimization; that is, it picks the lowest cost evaluation plan from among the various alternatives.

- **Query evaluation engine** -- executes low-level instructions generated by the DML compiler.

查询评估引擎

Query Processing

- 1. Parsing and translation
- 2. Optimization
- 3. Evaluation



事务

Transaction Management

- A **transaction** is a **collection of operations** that performs a single logical function in a database application
- Transaction-management component ensures that the database remains in a consistent (correct) state despite system failures (e.g., power failures and operating system crashes) and transaction failures.
- Concurrency-control manager controls the interaction among the concurrent transactions, to ensure the consistency of the database.

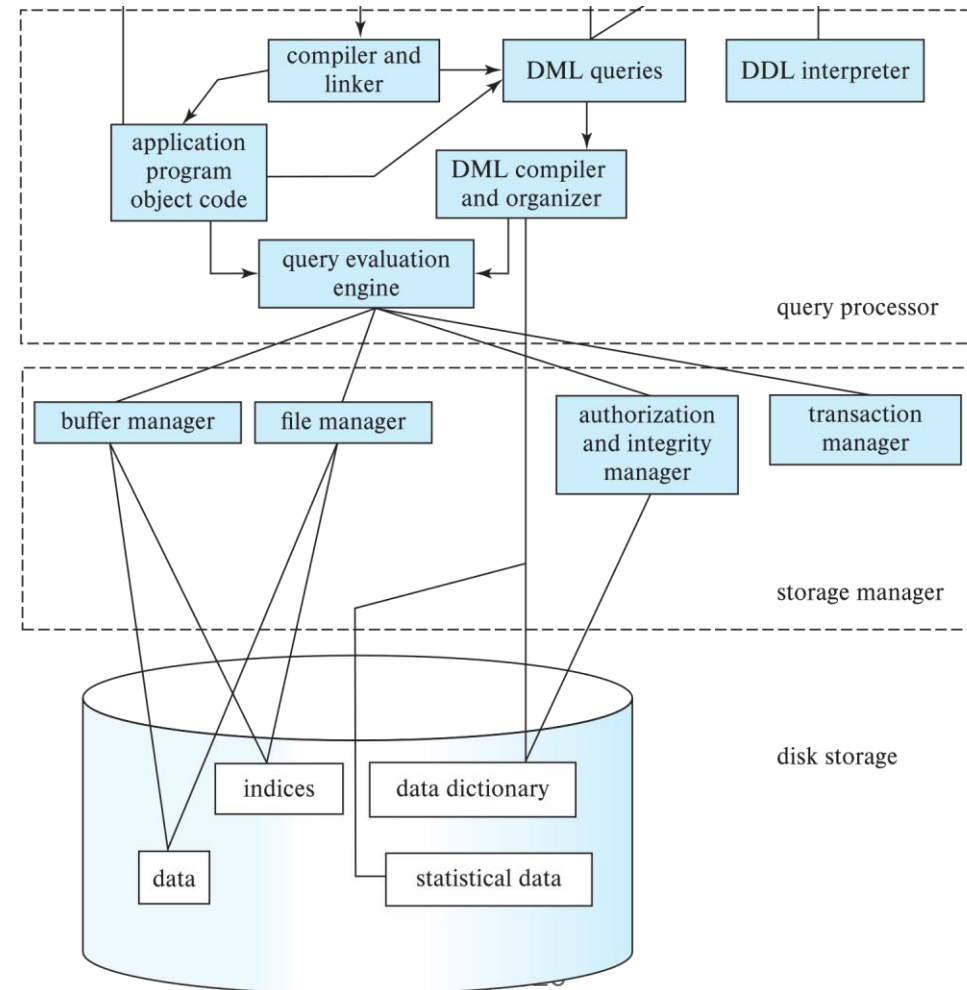
并发控制管理器

Database Architecture

数据库架构

- **Centralized databases**
 - One to a few cores, shared memory
- **Client-server,**
 - One server machine executes work on behalf of multiple client machines.
- **Parallel databases**
 - Many core shared memory
 - Shared disk
 - Shared nothing
- **Distributed databases**
 - Geographical distribution
 - Schema/data heterogeneity

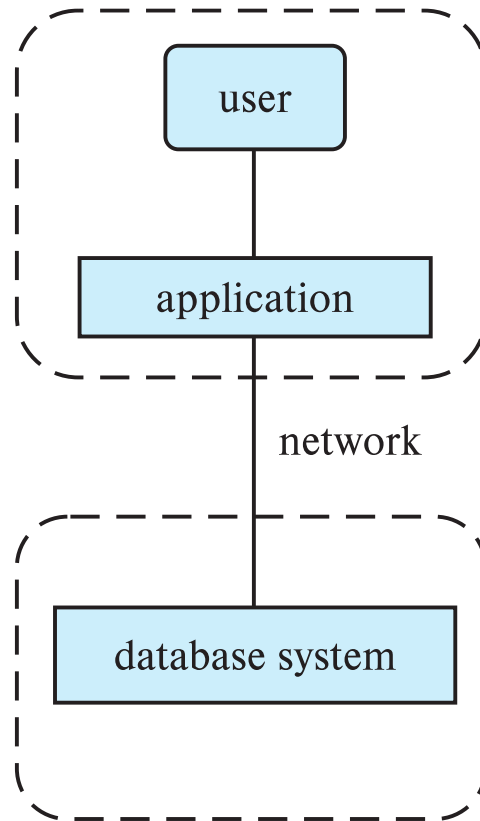
Database Architecture (Centralized/Shared-Memory)



Database Applications

- Database applications are usually partitioned into two or three parts
- **Two-tier architecture** -- the application resides at the client machine, where it invokes database system functionality at the server machine
- **Three-tier architecture** -- the client machine acts as a front end and does not contain any direct database calls.
 - The client end communicates with an application server, usually through a forms interface.
 - The application server in turn communicates with a database system to access data.

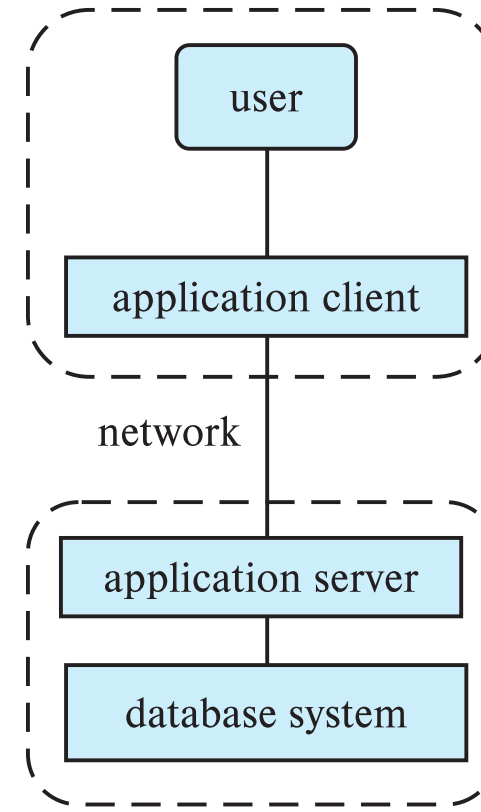
Two-tier and three-tier architectures



(a) Two-tier architecture

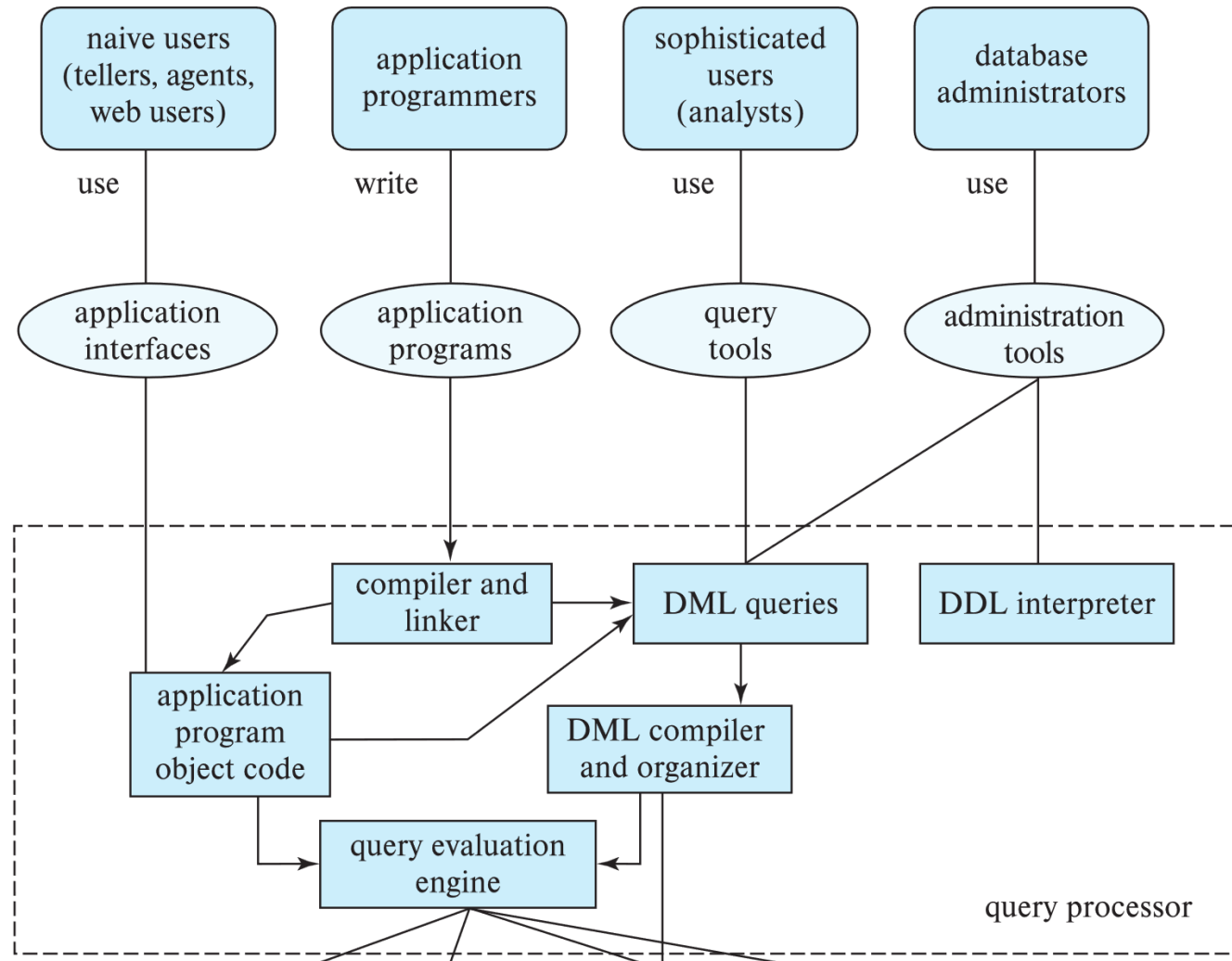
client

server



(b) Three-tier architecture

Database Users



Database Administrator

- **A person who has central control over the system is called a database administrator (DBA). Functions of a DBA include:**
 - **Schema definition**
 - **Storage structure and access-method definition**
 - **Schema and physical-organization modification**
 - **Granting of authorization for data access**
 - **Routine maintenance**
 - **Periodically backing up the database**
 - **Ensuring that enough free disk space is available for normal operations, and upgrading disk space as required**
 - **Monitoring jobs running on the database**

Course Info. & Logistics

Course Summary

- This course is designed to provide a comprehensive understanding of database **concepts, design, implementation, and management.**
- Databases are integral to modern computing and are critical for the storage, retrieval, and management of data in various applications ranging from web applications to large-scale enterprise systems.
- We will learn multiple topics, such as **data models and entity-relationship models, relational database design, SQL programming, indexing and query optimization, database security and administration, NoSQL databases, big data and distributed databases, etc.**

Topics Covered (Tentative)

- **Relational Languages, Structured Query Language (SQL)**
- **Database Design: The Entity-Relationship Model**
- **Relational Database Design**
- **Application Design and Development: Complex Data Types**
- **Data Analysis**
- **Data Storage Structures**
- **Indexing and Hashing**
- **Query Processing; Transactions**

Course Info.

- Date & Time: **Tuesday, 10:00-12:45**
- Location: **연암관 902**
- Office hour (On-Demand):
 - Time: **Mon. 17:00 - 17:30, Tue. 17:00 - 17:30**
 - Office: **팔달관 (Paldal Hall) 1011**
 - Also available by Zoom meeting.

Course Learning Platform



- Platform: Ajou Blackboard (Bb)
 - Webpage: <https://eclass2.ajou.ac.kr/ultra/institution-page>
 - Android and iOS apps available (<https://linktr.ee/ajoubb>).
 - Lecture slides, materials, quizzes, assignments, etc. will be updated.
 - *Recorded lectures* will be provided.
- Questions and Answers
 - Encourage to use the Ajou Bb to ask questions.
 - The instructor and the TA can try to answer your questions.
 - Your questions can also help others to improve their understandings.



QnA



Visible to students ▼

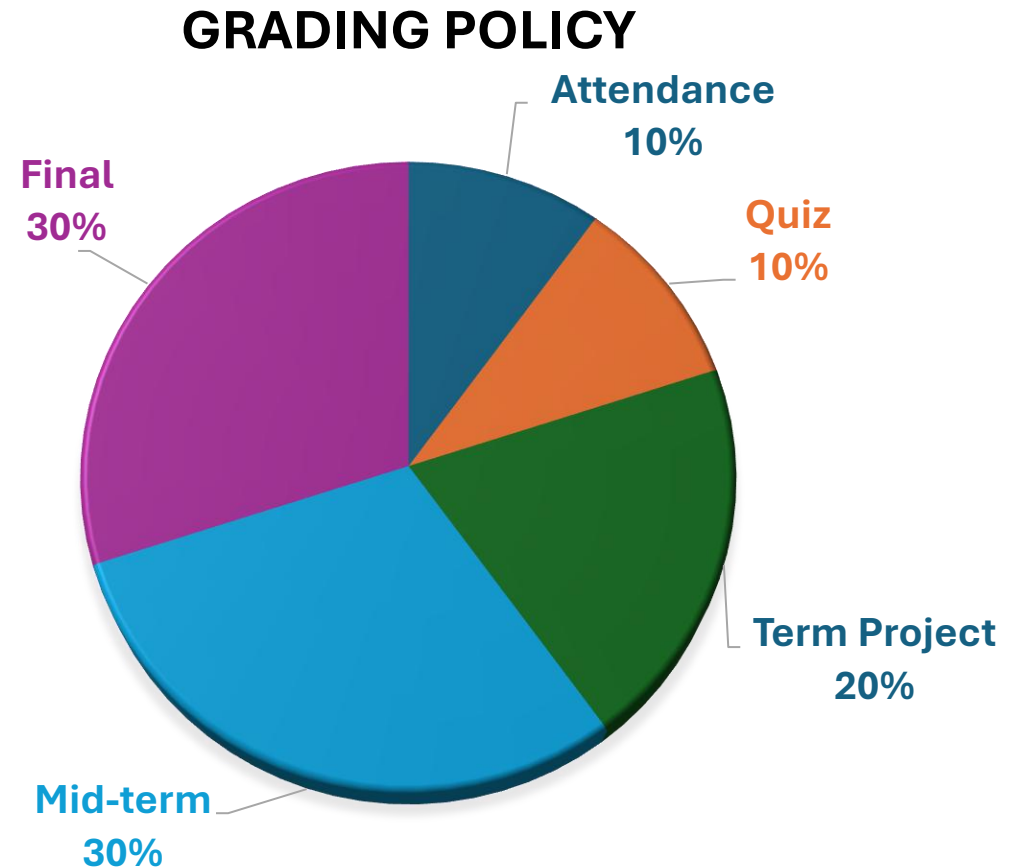
Please leave a question for this course.

Communications

- We use **Ajou Bb** and **email** for communications:
 - Please **specify the objective** of your questions.
 - ❑ **Good title**: How to use different parameters to read a csv file in python?
 - ❑ **Bad title**: Questions about the lecture
 - If using email, please clearly state **your name** and **student ID** in the email.
- Use **English/Chinese** for communications, questions in Ajou Bb, and all assignments.
 - If you use other languages in assignments and exams, you may expect to get **ZERO** points.

Grading Policy

- TA:
 - Email:
- Evaluation Means:
 - Quizzes;
 - Term Project;
 - Mid-term and final exams.
- Grading Policy:



Attendance

- Use the Electronic Attendance System.
 - Guideline: <https://www.ajou.ac.kr/kr/ajou/notice.do?mode=view&articleNo=192571&article.offset=0&articleLimit=10&srSearchVal=%EA%B3%84%EC%A0%88>
- 4 absences -> Fail
 - 2 lates/early leaves -> 1 absence
- Absent by reasonable causes with proof:
 - Sick, military training, family causes, etc. In detail: <https://www.ajou.ac.kr/kr/ajou/notice.do?mode=view&articleNo=324578&article.offset=0&articleLimit=10>
 - Submit a proper proof within 7 days after the absent in EAS.

Attendance in EAS



아주대학교 통합 모바일

전자출석부 사용법

학생증 태그 방식 사용 불가

블루투스 비콘 인증방법

1

앱 다운로드

스토어 검색 창에 '아주대학교 통합 모바일'을 입력하여 모바일 앱을 다운로드

2

앱 실행

블루투스 및 위치서비스를 반드시 ON으로 설정!

3

출석 인증 전

강의 시작 15분 전부터 출결 화면이 실행됩니다. 강의실 내부에서 '출석하기' 버튼을 눌러주세요.

4

출석 인증 후

사용자의 GPS를 분석하여 강의실 내부일 경우에만 출석인증이 가능합니다.

인증번호 출석방법

인증번호 출석

인증번호 출석을 진행하는 경우, '인증번호' 버튼이 활성화됩니다. 인증번호 입력창으로 이동!

마무리

교수님이 지정한 인증번호를 입력한 경우, 출석으로 처리됩니다.

자세한 매뉴얼은 학교 공지사항 참조 (포털-공지사항-전자출결로 검색)

- **Attended: 15 mins before and 10 mins after** the start time of a lecture.
 - **BLE-based** attendance checking by clicking in the smartphone App.
 - **Verification number-based** attendance checking.
- **Late: from 10 mins to 30 mins after** the start time.
- **Make sure to check your attendance status for each lecture in EAS.**
 - **Any undefined status (미인증) will be absent.**

Quiz and Assignment

- **Quizzes**

- Totally we will have **4** quizzes with each **10 questions**.
- Quiz will be given from **week 3 (once bi-weekly)**.
- **Format:**
 - Multiple choice single answer questions.

- **Term Project**

- We will have **1** term project.
- **Items to submit:**
 - Project reports
 - Source code (including documents of running instructions and code structures)
 - Presentation slides (powerpoint file)
 - A demo (video clip) if possible

Mid-term and Final Exams

- **Mid-term**
 - Format: **closed-book written exam in class**
 - Date: **April 14th, 10:00am – 11:00am**
 - Duration: **60 mins**
 - Allowed to bring an exam note: **1-side A4 handwritten**
 - **Must be handwritten by yourself**
 - **Not a copy from others**
- **Final Exam**
 - Format: **closed-book written exam in class**
 - Date: **June 9th, 10:00am – 11:00am**
 - Duration: **60 mins**
 - Allowed to bring an exam note: **1-side A4 handwritten**
 - **Must be handwritten by yourself**
 - **Not a copy from others**

Calendar

Calendar Grid (March 2026 to June 2025)

Legend:

- Blue: Regular day
- Red: Weekend
- Green: Public Holiday in Korea
- Pink: DBS Midterm / Final day

March 2026:

S	M	T	W	T	F	S
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	31				

April 2026:

S	M	T	W	T	F	S
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30		

May 2026:

S	M	T	W	T	F	S
					1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30
31						

June 2025:

S	M	T	W	T	F	S
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30				

Legend:

- Blue: Regular day
- Red: Weekend
- Green: Public Holiday in Korea
- Pink: DBS Midterm / Final day

Term Projects by Teams

- Using a mobility simulator to **design database**.
- “Simulation of Urban Mobility” (**SUMO**): <https://eclipse.dev/sumo/>
 - Microscopic and continuous traffic simulation
 - Doc: <https://sumo.dlr.de/docs/index.html>
- Using a Python interface to control the simulation, called TraCI (Traffic Control Interface).
- Using the Python interface with SQL codes to design a database.
- Each team shall consist of 2 students.

Term Project Topics (to be detailed)

- **Taxis (or shared-rides) information database system**
出租车（或共乘车辆）信息数据库系统
- **Shared-ride users information database system**
共乘用户信息数据库系统
- **Ride sharing platform management system**
共享乘车平台管理系统
- **Vehicle navigation information database system**
车辆导航信息数据库系统
- **Urban road traffic database system**
城市道路交通数据库系统

LLM-based Tools Use Policy

- **LLM-based tools** (e.g., ChatGPT and Gemini) are **NOT allowed** to **directly copy the generated content** in any Quizzes, Assignments, and Exams.
- In case of **term project report**, if any texts are generated or modified by an LLM tool, **a statement shall be given** at the end of the report to describe **where being modified** by **what kind of LLM tool** and make sure the modified text follow the original intention of writing.
 - Information to be given: **tool name, version, prompt inputs**

Academic Misconduct

- **Cheating**
 - Cheating includes fraud, deceit, or dishonesty in an academic assignment, or using or attempting to use materials, or assisting others in using materials that are prohibited or inappropriate in the context of the academic assignment in question.
- **Plagiarism**
 - Plagiarism includes use of intellectual material produced by another person without acknowledging its source.
- **False Information and Representation and Fabrication or Alteration of Information**
 - Furnishing false information, failing to identify oneself honestly, fabricating or altering information and presenting it as legitimate, or providing false or misleading information to an instructor or any other University official in an academic context.
- **Theft or Damage of Intellectual Property**
 - Sabotaging or stealing another person's work, improper access to or electronically interfering with the property of another person or the university, or obtaining a copy of an exam or assignment prior to its approved release.
- **Alteration of University Documents**
 - Forgery of an instructor's signature, submitting an altered transcript of grades to or from another institution or employer, putting one's name on another individual's work, or falsely altering a previously graded exam or assignment.

学术不端行为

- 作弊
 - 作弊包括在学术作业中欺诈、欺骗或不诚实的行为，或使用或试图使用相关材料，或协助他人使用与学术作业内容不符或不适当的材料。
- 剽窃
 - 剽窃包括使用他人创作的智力材料而不注明来源。
- 虚假信息、陈述及伪造或篡改信息
 - 提供虚假信息，未能诚实地表明身份，伪造或篡改信息并将其视为合法信息，或在学术环境中向教师或任何其他大学官员提供虚假或误导性信息。
- 盗窃或破坏知识产权
 - 破坏或窃取他人作品，不当获取或以电子方式干扰他人或大学的财产，或在考试或作业批准发布之前获取其副本。
- 篡改大学文件
 - 伪造教师签名、向其他机构或雇主提交篡改的成绩单、在他人作业上署名、或伪造已评分的考试或作业。

Database System

Thank you! Q/A?

Instructor: Yiwen Shen, Ph.D.

Dept. of Software & Computer Engineering,

Ajou University, Korea

chrisshen@ajou.ac.kr